

**IF2211 - Strategi Algoritma**

## **Tugas Kecil 3**

**Ice Sliding Puzzle Solver**



Disusun Oleh:

Mikhael Benrael Tampubolon - 13524009

**Program Studi Teknik Informatika  
Sekolah Teknik Elektro dan Informatika  
Institut Teknologi Bandung**

**2026**

## 1. ALGORITMA PATHFINDING YANG DIGUNAKAN

Program ini menggunakan empat algoritma pencarian (pathfinding), yaitu Uniform Cost Search (UCS), Greedy Best-First Search (GBFS), A\* (A-Star), dan Breadth-First Search (BFS) sebagai algoritma bonus. Papan permainan direpresentasikan sebagai matriks, di mana perpindahan dilakukan dengan meluncur lurus hingga menabrak rintangan atau target.

### A. Uniform Cost Search (UCS)

Algoritma ini melakukan pencarian dengan memprioritaskan rute yang memiliki total biaya (bobot lintasan) paling murah sejauh ini.

#### Langkah-langkah:

1. Masukkan posisi awal (Z) ke dalam Antrean Prioritas (Priority Queue) dengan status target angka 1 dan total biaya 0.

```
State initial = {start_pos.r, start_pos.c, 1, 0, 0, 0, ""};  
pq.push(initial);
```

2. Keluarkan status (state) yang memiliki total biaya pergerakan terkecil dari antrean.

```
State curr = pq.top();  
pq.pop();
```

3. Periksa apakah posisi saat ini berada di titik keluar (0) dan seluruh target angka sudah dikumpulkan secara berurutan. Jika ya, pencarian selesai.

```
if (grid[curr.r][curr.c] == '0' && curr.next_target > max_target_num) {  
    solution = curr; found = true; break; }
```

4. Jika tidak, simpan status tersebut ke dalam daftar riwayat (Closed-List) untuk mencegah perulangan rute mundur.

5. Melihat semua kemungkinan pergerakan (Atas, Bawah, Kiri, Kanan). Untuk setiap arah, simulasikan peluncuran aktor hingga ia berhenti

karena menabrak dinding (X), dan akumulasi bobot lintasan yang dilewatinya.

```
char dirs[] = {'U', 'D', 'L', 'R'}; int dr[] = {-1, 1, 0, 0}; int dc[] = {0, 0, -1, 1};  
for (int i = 0; i < 4; ++i) { pair<State, bool> res = slide(curr, dirs[i], dr[i],  
dc[i]); }
```

6. Jika rute sah, masukkan status baru tersebut ke dalam antrian dengan prioritas berdasarkan total biaya.
7. Ulangi langkah 2 hingga solusi ditemukan atau antrian kosong.

## B. Greedy Best-First Search (GBFS)

Algoritma ini sangat agresif dan memprioritaskan rute yang secara estimasi jarak (heuristik) berada paling dekat dengan target angka berikutnya.

### Langkah-langkah:

1. Masukkan posisi awal ke antrian prioritas dengan perhitungan nilai heuristik awal.

```
initial.heuristic = calculateHeuristic(initial.r, initial.c, 1, h_type);
```

2. Keluarkan status yang memiliki nilai heuristik terkecil (jarak terdekat ke target selanjutnya), tanpa mempedulikan seberapa besar biaya lintasan yang sudah dikeluarkan sebelumnya.
3. Cek kondisi menang. Jika belum, catat ke Closed-List.
4. Simulasikan peluncuran ke empat arah. Hitung nilai heuristik titik berhenti baru terhadap koordinat target angka selanjutnya menggunakan salah satu dari tiga metode: Manhattan, Euclidean, atau Chebyshev.

```
int dr = abs(r - t_pos.r); int dc = abs(c - t_pos.c); if (h_type == 1) return  
dr + dc; // Manhattan if (h_type == 2) return sqrt(dr * dr + dc * dc); //  
Euclidean if (h_type == 3) return max(dr, dc); // Chebyshev
```

5. Masukkan status baru tersebut ke antrian berdasarkan nilai heuristiknya.  
Ulangi

```
State initial = {start_pos.r, start_pos.c, 1, 0, 0, 0, ""};  
pq.push(initial);
```

### C. A\*

Algoritma ini menyeimbangkan eksplorasi dengan menggabungkan biaya aktual masa lalu dan estimasi jarak masa depan.

#### Langkah-langkah:

1. Masukkan posisi awal ke antrean prioritas.
2. Keluarkan status yang memiliki nilai evaluasi keseluruhan terkecil, yaitu hasil penjumlahan antara total biaya lintasan sejauh ini ditambah nilai heuristik jarak sisa ke target.

```
if (algo == "A*") { next_s.heuristic = calculateHeuristic(next_s.r,
next_s.c, next_s.next_target, h_type); next_s.f_value = next_s.cost +
next_s.heuristic; // f(n) = g(n) + h(n) }
```

3. Cek kondisi menang, lalu catat ke Closed-List.
4. Simulasikan peluncuran ke empat arah dan hitung total biaya lintasan serta jarak heuristik barunya.
5. Masukkan status tersebut ke antrean prioritas. Ulangi pencarian.

### D. Breadth-First Search (BFS)

Algoritma pencarian buta yang memprioritaskan jumlah tahapan pergerakan/luncuran paling sedikit.

#### Langkah-langkah:

1. Masukkan posisi awal ke antrean prioritas.
2. Keluarkan status yang memiliki panjang urutan langkah (path length) paling pendek.
3. Cek kondisi menang dan catat riwayat.
4. Simulasikan peluncuran ke empat arah. Biaya lintasan diabaikan sepenuhnya dalam prioritas antrean.

```
if (algo == "BFS") {
    next_s.f_value = next_s.path.length(); // f(n) = panjang path
}
```

5. Masukkan status baru ke antrean dengan prioritas jumlah langkah. Ulangi.

## 2. ANALISIS ALGORITMA PATHFINDING

### a. Definisi dari $f(n)$ dan $g(n)$ pada algoritma

- i.  $g(n)$ : Merupakan biaya aktual (*actual cost*) yang telah dikeluarkan untuk berpindah dari posisi awal ( $Z$ ) hingga mencapai titik atau *state*  $n$  saat ini. Pada permainan ini,  $g(n)$  adalah akumulasi dari angka-angka bobot ubin es yang secara fisik telah dilindas oleh aktor.
- ii.  $f(n)$ : Merupakan fungsi evaluasi utama yang menentukan urutan atau prioritas suatu *state* di dalam *Priority Queue*.
  1. Pada UCS,  $f(n) = g(n)$ .
  2. Pada GBFS,  $f(n) = h(n)$  (nilai heuristik murni).
  3. Pada A\*,  $f(n) = g(n) + h(n)$ .
  4. Pada BFS,  $f(n)$  = panjang dari string jalur pergerakan (contoh: "RDL" memiliki  $f(n) = 3$ ).

### b. Analisis Heuristik Pada Algoritma A\*

Heuristik yang digunakan, yaitu Manhattan, Euclidean, dan Chebyshev, termasuk admissible karena nilai perkiraannya tidak pernah melebihi biaya sebenarnya menuju tujuan. Pada permainan ini, aktor bergerak dengan cara meluncur sehingga harus melewati beberapa petak sebelum berhenti. Perkiraan biaya minimum diasumsikan terjadi ketika aktor dapat bergerak lurus atau diagonal tanpa hambatan dan setiap petak memiliki bobot terkecil, yaitu 1. Ketiga heuristik tersebut hanya menghitung jarak secara geometris berdasarkan posisi awal dan tujuan. Oleh karena itu, nilai heuristik selalu lebih kecil atau sama dengan biaya nyata perjalanan, sehingga algoritma A\* tetap dapat menemukan solusi optimal.

### c. Perbandingan Algoritma yang Diimplementasikan

- i. **UCS**: Selalu menjamin rute dengan biaya paling minimum, namun sangat lambat dan boros memori karena melakukan ekspansi rute secara buta ke segala arah tanpa panduan arah target.
- ii. **GBFS**: Bekerja sangat cepat dan memori yang efisien karena langsung mengarah ke titik target. Namun, tidak menjamin solusi yang dihasilkan memiliki biaya terendah, dan rentan mengambil jalan memutar yang memakan biaya besar jika terhalang dinding.

- iii. **A\***: Algoritma ini menjamin rute dengan biaya paling minimum layaknya UCS, namun memangkas jumlah iterasi pencarian secara signifikan karena dituntun oleh arah lintasan seperti GBFS.
- iv. **BFS**: Menjamin solusi dengan jumlah langkah paling sedikit, namun sama sekali tidak memedulikan bobot lintasan. Algoritma ini akan memakan waktu pencarian yang sangat tinggi pada papan yang luas.

### 3. SOURCE PROGRAM

#### a. IceSlider.h

```
#ifndef ICE_SLIDER_H
#define ICE_SLIDER_H

#include <iostream>
#include <vector>
#include <string>
#include <fstream>
#include <queue>
#include <chrono>
#include <cmath>
#include <algorithm>

using namespace std;

struct Point {
    int r, c;
};

struct State {
    int r, c;
    int next_target;
```

```

    int cost;
    int heuristic;
    int f_value;
    string path;

    bool operator>(const State& other) const {
        return f_value > other.f_value;
    }
};

class IceSlider {
private:
    vector<string> grid;
    vector<vector<int>> cost_grid;
    int rows, cols;
    Point start_pos;
    Point target_positions[10];
    int max_target_num;
    string loaded_filename;
public:
    IceSlider();
    bool loadMap(const string& filename);
    int calculateHeuristic(int r, int c, int next_target, int h_type);
    pair<State, bool> slide(State curr, char dir, int dr, int dc);
    void solve(string algo, int h_type);
    void runPlayback(const string& path);
};

#endif

```

b. IceSlide.cpp

```

#include "IceSlider.h"

```

```

IceSlider::IceSlider() : max_target_num(0) {}

bool IceSlider::loadMap(const string& filename) {
    ifstream file(filename);
    if (!file.is_open()) return false;

    file >> rows >> cols;
    grid.resize(rows);
    cost_grid.assign(rows, vector<int>(cols, 0));

    for (int i = 0; i < rows; ++i) {
        file >> grid[i];
        for (int j = 0; j < cols; ++j) {
            if (grid[i][j] == 'Z') start_pos = {i, j};
            else if (isdigit(grid[i][j])) {
                int num = grid[i][j] - '0';
                target_positions[num] = {i, j};
                if (num > 0) max_target_num = max(max_target_num, num);
            }
        }
    }

    for (int i = 0; i < rows; ++i) {
        for (int j = 0; j < cols; ++j) {
            file >> cost_grid[i][j];
        }
    }
    file.close();
    loaded_filename = filename;
    return true;
}

```



```

int IceSlider::calculateHeuristic(int r, int c, int next_target, int h_type) {
    Point t_pos;
    if (next_target <= max_target_num) t_pos = target_positions[next_target];
    else t_pos = target_positions[0];

    int dr = abs(r - t_pos.r);
    int dc = abs(c - t_pos.c);

    if (h_type == 1) return dr + dc;
    if (h_type == 2) return sqrt(dr * dr + dc * dc);
    if (h_type == 3) return max(dr, dc);
    return 0;
}

pair<State, bool> IceSlider::slide(State curr, char dir, int dr, int dc) {
    int r = curr.r, c = curr.c;
    int move_cost = 0;
    int current_target = curr.next_target;
    bool isValid = true;

    while (true) {
        int nr = r + dr;
        int nc = c + dc;

        if (nr < 0 || nr >= rows || nc < 0 || nc >= cols) { isValid = false; break; }

        char tile = grid[nr][nc];
        if (tile == 'X') break;
        if (tile == 'L') { isValid = false; break; }

        r = nr; c = nc;
        move_cost += cost_grid[r][c];
    }
}

```

```

        if (isdigit(tile)) {
            int val = tile - '0';
            if (val > 0 && val == current_target) {
                current_target++;
            } else if (val > 0 && val > current_target) {
                isValid = false; break;
            }
        }
    }
}

if (isValid && grid[r][c] == '0' && current_target > max_target_num) {}

State next_state = {r, c, current_target, curr.cost + move_cost, 0, 0, curr.path
+ dir};
return {next_state, isValid};
}

void IceSlider::solve(string algo, int h_type) {
    priority_queue<State, vector<State>, greater<State>> pq;

    vector<pair<string, int>> best_cost_list;

    auto start_time = chrono::high_resolution_clock::now();
    int iterations = 0;

    State initial = {start_pos.r, start_pos.c, 1, 0, 0, 0, ""};
    if (algo != "BFS") initial.heuristic = calculateHeuristic(initial.r, initial.c, 1,
h_type);
    pq.push(initial);

    State solution;
    bool found = false;

```

```

vector<string> log_iterasi;

while (!pq.empty()) {
    State curr = pq.top();
    pq.pop();
    iterations++;

    string current_path = curr.path.empty() ? "START" : curr.path;
    string log_text = "Iterasi " + to_string(iterations) +
        " | Posisi: (" + to_string(curr.r) + "," + to_string(curr.c) + ")" +
        " | Next Target: " + to_string(curr.next_target) +
        " | Cost: " + to_string(curr.cost) +
        " | Path: " + current_path;
    log_iterasi.push_back(log_text);
    if (grid[curr.r][curr.c] == '0' && curr.next_target > max_target_num) {
        solution = curr;
        found = true;
        break;
    }

    string state_key = to_string(curr.r) + "," + to_string(curr.c) + "," +
to_string(curr.next_target);

    bool skip_state = false;
    bool found_key = false;

    for (int i = 0; i < best_cost_list.size(); i++) {
        if (best_cost_list[i].first == state_key) {
            found_key = true;
            if (best_cost_list[i].second <= curr.cost) {
                skip_state = true;
            } else {
                best_cost_list[i].second = curr.cost;
            }
        }
    }
}

```

```

        break;
    }
}

if (skip_state) continue;
if (!found_key) {
    best_cost_list.push_back({state_key, curr.cost});
}

char dirs[] = {'U', 'D', 'L', 'R'};
int dr[] = {-1, 1, 0, 0};
int dc[] = {0, 0, -1, 1};

for (int i = 0; i < 4; ++i) {
    pair<State, bool> res = slide(curr, dirs[i], dr[i], dc[i]);
    if (res.second) {
        State next_s = res.first;

        if (algo == "UCS") { next_s.f_value = next_s.cost; }
        else if (algo == "GBFS") { next_s.heuristic =
calculateHeuristic(next_s.r, next_s.c, next_s.next_target, h_type);
next_s.f_value = next_s.heuristic; }
        else if (algo == "A*") { next_s.heuristic = calculateHeuristic(next_s.r,
next_s.c, next_s.next_target, h_type); next_s.f_value = next_s.cost +
next_s.heuristic; }
        else if (algo == "BFS") { next_s.f_value = next_s.path.length(); }

        pq.push(next_s);
    }
}

auto end_time = chrono::high_resolution_clock::now();

```

```

    auto duration_micro =
chrono::duration_cast<chrono::microseconds>(end_time - start_time);
    if (found) {
        cout << "\nSolusi" << endl;
        cout << "Path      : " << solution.path << endl;
        cout << "Total Cost  : " << solution.cost << endl;
        cout << "Iterasi    : " << iterations << endl;
        cout << "Waktu (ms) : " << duration_micro.count() << " microsecond ("
<< duration_micro.count() / 1000.0 << " ms)\n" << endl;

        cout << ">> Apakah Anda ingin melakukan playback? (Ya/Tidak): ";
        string play; cin >> play;
        if (play == "Ya" || play == "ya") {
            runPlayback(solution.path);
        }

        cout << ">> Apakah Anda ingin menyimpan solusi? (Ya/Tidak): ";
        string saveOpt; cin >> saveOpt;
        if (saveOpt == "Ya" || saveOpt == "ya") {
            string base_name = loaded_filename;
            size_t pos = base_name.find_last_of("\\");
            if (pos != string::npos) {
                base_name = base_name.substr(pos + 1);
            }
            string savePath = "solve/solusi_" + base_name;
            ofstream outFile(savePath);

            if (outFile.is_open()) {
                outFile << "Riwayat Iterasi\n";
                for (const string& log : log_iterasi) {
                    outFile << log << "\n";
                }
                outFile << "Path: " << solution.path << "\n";
            }
        }
    }
}

```

```

        outFile << "Cost: " << solution.cost << "\n";
        outFile << "Total Iterasi: " << iterations << "\n";
        outFile << "Waktu: " << duration_micro.count() << " microsecond ("
            << duration_micro.count() / 1000.0 << " ms)\n";

        outFile.close();
        cout << ">> Berhasil disimpan di" << savePath << "\n";
    } else {
        cout << ">> Gagal menyimpan solusi.\n";
    }
}
} else {
    cout << "Solusi tidak ditemukan." << endl;
}
}

void IceSlider::runPlayback(const string& path) {
    vector<vector<string>> frames;
    vector<string> temp_grid = grid;
    int r = start_pos.r, c = start_pos.c;
    int current_target = 1;

    frames.push_back(temp_grid);

    for (char dir : path) {
        int dr = 0, dc = 0;
        if (dir == 'U') dr = -1; else if (dir == 'D') dr = 1;
        else if (dir == 'L') dc = -1; else if (dir == 'R') dc = 1;

        temp_grid[r][c] = '*';

        while (true) {
            int nr = r + dr, nc = c + dc;

```

```

        if (temp_grid[nr][nc] == 'X') break;
        r = nr; c = nc;

        if (isdigit(temp_grid[r][c])) {
            int val = temp_grid[r][c] - '0';
            if (val > 0 && val == current_target) {
                temp_grid[r][c] = '*';
                current_target++;
            }
        }
        temp_grid[r][c] = 'Z';
        frames.push_back(temp_grid);
    }

    int step = 0;
    int max_step = frames.size() - 1;
    while (true) {
#ifdef _WIN32
        system("cls");
#else
        system("clear");
#endif

        cout << "Step " << step << " / " << max_step << endl;
        for (const string& row : frames[step]) cout << row << endl;

        cout << "\n[A] Mundur [D] Maju [J] Jump [Q] Keluar: ";
        char cmd; cin >> cmd;
        cmd = toupper(cmd);

        if (cmd == 'A' && step > 0) step--;
        else if (cmd == 'D' && step < max_step) step++;
    }
}

```

```

        else if (cmd == 'J') {
            cout << "Lompat ke step (0-" << max_step << "): ";
            int j; cin >> j;
            if (j >= 0 && j <= max_step) step = j;
        }
        else if (cmd == 'Q') break;
    }
}

```

c. main.cpp

```

#include "IceSlider.h"

int main() {
    IceSlider game;
    string filename;

    cout << ">> Masukan file input :\n";
    cin >> filename;

    if (!game.loadMap(filename)) {
        cout << "Gagal membaca file " << filename << endl;
        return 1;
    }

    cout << ">> Algoritma apa yang anda pilih? (UCS/GBFS/A*/BFS)\n";
    string algo; cin >> algo;

    int h_type = 1;
    if (algo == "A*" || algo == "GBFS") {
        cout << ">> Heuristic apa yang anda pilih?\n";
        cout << "1. Manhattan (H1)\n2. Euclidean (H2)\n3. Chebyshev (H3)\n";
        cout << "Pilihan (1/2/3): ";
    }
}

```



```

    cin >> h_type;
}

game.solve(algo, h_type);
return 0;
}

```

#### 4. TANGKAPAN LAYAR INPUT & OUTPUT

##### a. input1.txt

```

7 7
XXXXXXX
X0***X
X**X**X
X****OX
X1***LX
XZ**X*X
XXXXXXX
999 999 999 999 999 999 999
999 3 5 2 8 1 999
999 7 4 999 6 9 999
999 2 8 3 5 4 999
999 6 1 7 2 999 999
999 9 3 4 999 8 999
999 999 999 999 999 999 999

```

Dengan A\* & Euclidean

```

PS C:\Users\Asus\OneDrive\Documents\Tucil3_13524
test/input1.txt
>> Algoritma apa yang anda pilih? (UCS/GBFS/A*/B
A*
>> Heuristic apa yang anda pilih?
1. Manhattan (H1)
2. Euclidean (H2)
3. Chebyshev (H3)
Pilihan (1/2/3): 2

=== Solusi Ditemukan ===
Path          : U
Total Cost    : 18
Iterasi       : 7
Waktu (ms)    : 79 microsecond (0.079 ms)

```

b. input2.txt

```

5 5
XXXXX
XZ1*X
X***X
XX0*X
XXXXX
999 999 999 999 999
999 1 1 1 999
999 1 1 1 999
999 999 1 1 999
999 999 999 999 999

```

Dengan UCS

```

>> Masukan file input :
test/input2.txt
>> Algoritma apa yang anda pilih? (UCS/GBFS/A*/BFS)
UCS

=== Solusi Ditemukan ===
Path          : RDL
Total Cost    : 5
Iterasi       : 25
Waktu (ms)    : 273 microsecond (0.273 ms)

```

c. input3.txt

```

6 6
XXXXXX
XZ*1*X
XXXX*X
X0***X
X2***X
XXXXXX
99 99 99 99 99 99
99 1 1 1 1 99
99 99 99 99 1 99
99 1 1 1 1 99
99 1 1 1 1 99
99 99 99 99 99 99

```

Dengan GBFS & Manhattan

```

>> Masukkan file input :
test/input3.txt
>> Algoritma apa yang anda pilih? (UCS/GBFS/A*/BFS)
GBFS
>> Heuristic apa yang anda pilih?
1. Manhattan (H1)
2. Euclidean (H2)
3. Chebyshev (H3)
Pilihan (1/2/3): 1

=== Solusi Ditemukan ===
Path          : RDLU
Total Cost    : 10
Iterasi       : 8
Waktu (ms)    : 116 microsecond (0.116 ms)

```

d. input4.txt

```

7 7
XXXXXXX
XZ***1X
XLXXX*X
X*2***X
X*XLXXX
X0***X
XXXXXXX
999 999 999 999 999 999 999
999 1 1 1 1 1 999
999 999 999 999 999 1 999
999 1 1 1 1 1 999
999 1 999 999 999 999 999
999 1 1 1 1 1 999
999 999 999 999 999 999 999

```

Dengan UCS

```
>> Masukan file input :
test/input4.txt
>> Algoritma apa yang anda pilih? (UCS/GBFS/A*/BFS)
BFS

=== Solusi Ditemukan ===
Path          : RDLD
Total Cost    : 12
Iterasi       : 17
Waktu (ms)    : 169 microsecond (0.169 ms)
```

e. input5.txt

```
10 10
XXXXXXXXXX
XZ*****X
X*X*****X
X*3***4X*X
X*****X
X*X0***X*X
X*****X*X
X*****X
X2*****1X
XXXXXXXXXX
999 999 999 999 999 999 999 999 999 999
999 5 5 5 5 5 5 5 5 999
999 5 999 1 1 1 1 1 5 999
999 5 5 5 5 5 5 999 5 999
999 5 1 1 1 1 5 1 5 999
999 5 999 5 5 5 5 1 5 999
999 5 1 1 1 1 999 1 5 999
999 5 1 1 1 1 1 1 5 999
999 5 5 5 5 5 5 5 5 999
999 999 999 999 999 999 999 999 999 999
```

Dengan BFS

```
>> Masukkan file input :  
test/input5.txt  
>> Algoritma apa yang anda pilih? (UCS/GBFS/A*/BFS)  
BFS  
Solusi tidak ditemukan.
```

## 5. ANALISIS HASIL PERCOBAAN

### a. input1.txt

Algoritma A\* dengan heuristik Euclidean berhasil mengeksekusi papan di mana titik awal (Z), target angka (1), dan pintu keluar (0) berada pada satu garis vertikal yang sama. A\* menyadari bahwa aktor dapat mengumpulkan target 1 dan langsung melesat ke angka 0 lalu berhenti menabrak dinding hanya dalam 1 kali luncuran (Path: U). Waktu eksekusinya sangat efisien (0.079 ms) meskipun harus memeriksa 7 kemungkinan *state* di dalam memorinya sebelum yakin bahwa rute tunggal U adalah yang termurah.

### b. Input2.txt

Pada uji coba ini, UCS digunakan untuk mencari biaya paling murah. Sifat pencarian buta dari UCS terlihat jelas di mana iterasi pencariannya membengkak hingga 25 iterasi untuk matriks yang sangat kecil (5x5). UCS harus menghitung dan merayapi berbagai rute lain sebelum menetapkan bahwa lintasan Kanan, Bawah, Kiri (RDL) adalah lintasan paling murah (Total Cost: 5).

### c. input3.txt

GBFS dengan jarak Manhattan membuktikan kecepatan agresifnya. Program hanya butuh membongkar 8 *state* iterasi untuk menemukan solusi akhir. Ini jauh lebih efisien secara memori dan waktu komputasi dibandingkan pencarian buta, karena GBFS hanya berfokus pada titik koordinat target terdekat. Meskipun cepat, biaya akhirnya bergantung murni pada kebetulan lintasan langsung tanpa evaluasi harga ubin di masa lalu.

### d. input4.txt

Implementasi Breadth-First Search menemukan solusi dalam 4 tahapan gerak (RDLD). BFS menjelajahi hierarki pergerakan level demi level, sehingga ia memastikan bahwa rute 4 tahap ini adalah rute dengan manuver paling sedikit

yang mungkin dilakukan. Walau begitu, biaya 12 yang dihasilkan tidak terjamin sebagai *cost* termurah karena BFS tidak mempertimbangkan besaran angka ubin dalam fungsi prioritasnya.

**e. input5.txt**

BFS gagal menemukan solusi pada peta "jebakan" 10x10 ini. Hal ini terjadi karena arsitektur BFS yang memprioritaskan urutan langkah dipadukan dengan implementasi pemotongan rute pada *Closed-List* yang berbasis *Cost*. Pada peta ini, terdapat jalur spiral berbiaya mahal yang wajib dilalui agar bisa menang, dan jalur buntu di tengah yang berbiaya murah. Saat BFS mengabaikan *cost* dan merayap berdasarkan langkah, status rute panjang yang berpotensi benar justru di-*pruned* dari ingatan oleh program karena sebelumnya *state* koordinat tersebut telah sempat dikunjungi oleh lintasan lain yang *cost*-nya secara kebetulan lebih murah, sehingga *queue* akhirnya kosong sebelum target ditemukan (State-space exhaustion).

## 6. Pranala Repository

[https://github.com/MikhaelBenrael/Tucil3\\_13524009.git](https://github.com/MikhaelBenrael/Tucil3_13524009.git)

## 7. Pernyataan dan Lampiran

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri.



Mikhael Benrael Tampubolon

| No | Poin                                          | Ya | Tidak |
|----|-----------------------------------------------|----|-------|
| 1  | Program berhasil di kompilasi tanpa kesalahan | ✓  |       |

|   |                                                                                                         |   |   |
|---|---------------------------------------------------------------------------------------------------------|---|---|
| 2 | Program berhasil dijalankan                                                                             | ✓ |   |
| 3 | Solusi yang diberikan program benar dan mematuhi aturan permainan                                       | ✓ |   |
| 4 | Program dapat membaca masukan berkas .txt serta menyimpan solusi dalam berkas .txt                      | ✓ |   |
| 5 | Program memiliki Graphical User Interface (GUI)                                                         |   | ✓ |
| 6 | Ada algoritma <i>pathfinding</i> alternatif selain dari spesifikasi wajib (Algoritma X dan Algoritma Y) | ✓ |   |
| 7 | Ada tambahkan dua alternatif heuristik selain yang utama (Heuristik X dan Heuristik Y)                  | ✓ |   |